

O QUE FOI FEITO

Fase 2 — Implementação da Plataforma AILO

Artificial Intelligence in a Learning Organization

Projeto de Engenharia Informática — Universidade Aberta

Ricardo & Carla

Baseado no framework de Santos & Mamede (2026)

Maio 2026

Índice

1. Enquadramento e Objetivos
2. Arquitetura Implementada
3. Backend — Modelos de Dados
4. Backend — API RESTful
5. Backend — Motor de Scoring
6. Backend — Assistente IA (Gemini)
7. Frontend — Interface de Utilizador
8. Base de Dados — Seed Data
9. Segurança e Autenticação
10. Testes e Validação
11. Instruções de Instalação e Execução
12. Instruções para Deployment Real
13. Trabalho Futuro

1. Enquadramento e Objetivos

A Fase 2 do projeto AILO teve como objetivo transformar toda a documentação conceptual e técnica produzida na Fase 1 numa plataforma web funcional (MVP — Minimum Viable Product). O AILO é uma plataforma de diagnóstico de maturidade organizacional para integração de Inteligência Artificial nas Organizações Aprendentes, baseada no framework teórico de Santos & Mamede (2026). O projeto é desenvolvido por Ricardo e Carla no âmbito do curso de Engenharia Informática.

O framework AILO organiza-se em 6 camadas interdependentes:

- **Organizacional** — Estratégia, processos, decisão e criação de valor
- **Humana** — Pessoas, cultura, autonomia e ética
- **Aprendizagem** — Contextos adaptativos, experiências personalizadas
- **Cognitiva (IA)** — Geração, recomendação, síntese e previsão
- **Tecnológica** — Plataformas, dados, integração, segurança
- **Avaliação** — Evidências, avaliação formativa, competências, impacto

A plataforma implementa 51 indicadores de maturidade distribuídos por 23 componentes nestas 6 camadas, cada um avaliado numa escala de 1 (Inicial) a 5 (Otimizado).

2. Arquitetura Implementada

A arquitetura segue o modelo de 3 camadas definido na Fase 1:

Frontend: HTML5, CSS3 (design system premium dark theme), JavaScript Vanilla, Chart.js para gráficos radar e barras.

Backend: Python 3 com Flask (app factory pattern), Flask-SQLAlchemy para ORM, Flask-CORS para CORS, PyJWT para autenticação JWT, bcrypt para hashing de passwords.

Base de Dados: SQLite (desenvolvimento), com modelo preparado para PostgreSQL em produção.

IA: Integração com Google Gemini API para assistente conversacional contextualizado.

2.1 Estrutura de Pastas

- Fase_2/backend/app/__init__.py — Flask app factory
- Fase_2/backend/app/config.py — Configurações dev/prod
- Fase_2/backend/app/models/ — 8 ficheiros de modelos SQLAlchemy
- Fase_2/backend/app/routes/ — 8 ficheiros de rotas API REST
- Fase_2/backend/app/services/ — Scoring, interdependências, IA, relatórios
- Fase_2/backend/app/utils/ — Auth JWT, decoradores, validadores
- Fase_2/backend/seeds/ — Dados iniciais (6 camadas, 23 componentes, 51 indicadores)
- Fase_2/frontend/ — HTML pages, CSS design system, JS modules

3. Backend — Modelos de Dados

Foram implementados os seguintes modelos SQLAlchemy, correspondentes às 12 tabelas definidas no modelo relacional da Fase 1:

Tabela	Descrição	Campos Principais
utilizadores	Utilizadores da plataforma	nome, email, password_hash, papel
organizacoes	Organizações a avaliar	nome, setor, dimensao, tipo, pais
camadas_ailo	6 camadas do framework	nome, descricao, peso, ordem, cor
componentes	23 componentes por camada	nome, camada_id, peso, ordem
indicadores	51 indicadores de maturidade	codigo, pergunta, desc_nivel_1/3/5, peso
avaliacoes	Instâncias de avaliação	organizacao_id, status, score_global, nivel
respostas	Respostas aos indicadores	avaliacao_id, indicador_id, score, justificacao
resultados_camada	Scores por camada	score, nivel, pontos_fortes, lacunas, recomendacoes
interdependencias	Relações entre camadas	camada_a/b, tipo_relacao, impacto
ferramentas_ia	Catálogo de ferramentas IA	nome, area_funcional, custo, complexidade
recomendacoes	Ferramentas recomendadas	ferramenta_id, prioridade, justificacao
conversas_ia	Histórico do chat IA	papel, mensagem, camada_id
relatorios	Relatórios gerados	titulo, conteudo_json, pdf_path

4. Backend — API RESTful

A API segue a especificação definida na Fase 1, com base URL /api/v1. Todos os endpoints (exceto auth) requerem token JWT no header Authorization.

Módulo	Método	Endpoint	Descrição
Auth	POST	/auth/register	Registo de utilizador
Auth	POST	/auth/login	Login com JWT
Auth	GET	/auth/me	Perfil do utilizador autenticado
Organizações	GET/POST	/organizacoes	Listar/Criar organizações
Organizações	GET/PUT/DEL	/organizacoes/:id	Detalhar/Atualizar/Eliminar
AILO	GET	/ailo/camadas	Listar camadas com componentes e indicadores
AILO	GET	/ailo/camadas/:id/indicadores	Indicadores de uma camada
Avaliações	POST/GET	/avaliacoes	Criar/Listar avaliações
Avaliações	GET	/avaliacoes/:id	Detalhar com progresso por camada
Avaliações	POST	/avaliacoes/:id/finalizar	Finalizar e calcular scoring
Respostas	POST	/avaliacoes/:id/respostas	Guardar resposta individual
Respostas	POST	/avaliacoes/:id/respostas/batch	Guardar múltiplas respostas
Resultados	GET	/avaliacoes/:id/resultados	Obter scores e interdependências
Chat IA	POST	/avaliacoes/:id/chat	Enviar mensagem ao assistente
Relatórios	POST	/avaliacoes/:id/relatorio	Gerar relatório

5. Backend — Motor de Scoring

O motor de scoring é o componente central da plataforma, implementado em `services/scoring.py`. Calcula a maturidade organizacional seguindo a hierarquia:

- 1. Score por Indicador:** Valor de 1-5 atribuído pelo avaliador, multiplicado pelo peso do indicador.
- 2. Score por Componente:** Média ponderada dos scores dos indicadores do componente.
- 3. Score por Camada:** Média ponderada dos scores dos componentes da camada.
- 4. Score Global:** Média ponderada dos scores das 6 camadas (pesos: Humana=1.2, Tecnológica=0.8, restantes=1.0).

Classificação de Níveis de Maturidade:

Score	Nível
1.0 - 1.8	Inicial
1.9 - 2.6	Em Desenvolvimento
2.7 - 3.4	Definido
3.5 - 4.2	Gerido
4.3 - 5.0	Otimizado

Adicionalmente, o serviço `interdependencias.py` analisa 4 pares críticos de camadas (ex: Humana×Avaliação, Cognitiva×Tecnológica), classificando a relação como fortalece, risco, bloqueia ou oportunidade, com mapeamento aos critérios CR1-CR6.

6. Backend — Assistente IA (Google Gemini)

O assistente IA está implementado em `services/ia_assistant.py` e utiliza a API do Google Gemini (modelo `gemini-2.0-flash` por defeito). Funcionalidades:

System prompt AILO: Instruções que contextualizam o modelo como especialista em aprendizagem organizacional e IA.

Contextualização dinâmica: O prompt inclui informações da organização (setor, dimensão, tipo) e da camada atual do questionário.

Histórico de conversa: As últimas 10 mensagens são incluídas para manter coerência conversacional.

Modo offline: Quando não há chave API configurada, o assistente responde com mensagem informativa em vez de erro.

Persistência: Todas as mensagens são guardadas na tabela `conversas_ia` para referência futura.

Nota importante: Para ativar o assistente IA completo, é necessário configurar a variável `GEMINI_API_KEY` no ficheiro `.env` com uma chave válida da Google AI Platform.

7. Frontend — Interface de Utilizador

O frontend foi desenvolvido em HTML5, CSS3 e JavaScript Vanilla, seguindo os wireframes definidos na Fase 1. Utiliza um design system premium com tema escuro.

7.1 Design System (main.css)

- Paleta de cores AILO (6 cores por camada + tons escuros premium)
- Tipografia Inter (Google Fonts) com pesos 300-800
- Componentes: cards, botões, formulários, tabelas, modais, toasts
- Glassmorphism na navbar (backdrop-filter blur)
- Micro-animações (fadeIn, slideIn, pulse, spin)
- Layout responsivo com breakpoint mobile (768px)

7.2 Páginas Implementadas

- **index.html** — Landing page com hero, features e hexágono AILO
- **login.html** — Formulário de autenticação com validação
- **register.html** — Registo de novos utilizadores
- **dashboard.html** — Painel principal com organizações, avaliações e gráfico radar (Chart.js)
- **organizacoes.html** — CRUD de organizações com modal de criação/edição
- **questionario.html** — Questionário AILO com tabs por camada, seleção 1-5, auto-save, barra de progresso e chat IA lateral
- **resultados.html** — Dashboard de resultados com radar hexagonal, barras por camada, pontos fortes/lacunas/recomendações e tabela de interdependências

8. Base de Dados — Seed Data

O ficheiro `seeds/seed_data.py` contém todos os dados iniciais do framework AILO, totalizando:

- **6 camadas** com nomes em PT/EN, descrições, pesos e cores
- **23 componentes** distribuídos pelas camadas
- **51 indicadores** com código único, pergunta em português, descrições de nível 1/3/5 e pesos
- **10 ferramentas IA** no catálogo inicial (Gemini, ChatGPT, Moodle, etc.)

O script `seed.py` popula a base de dados com estes dados. Pode ser executado múltiplas vezes (limpa e re-popula automaticamente).

9. Segurança e Autenticação

- **JWT (JSON Web Tokens):** Tokens com expiração de 24 horas, gerados via PyJWT.
- **Bcrypt:** Hashing de passwords com 12 rounds de salt.
- **Decoradores:** `@login_required` e `@admin_required` protegem todas as rotas da API.
- **Validação:** Campos obrigatórios, formato de email e sanitização de HTML.
- **CORS:** Configurado para aceitar pedidos da origem do frontend.
- **.env:** Chaves API e segredos armazenados em variáveis de ambiente, nunca no código.

10. Testes e Validação

A validação da Fase 2 incluiu:

- Execução do `seed.py` — 6 camadas, 23 componentes, 51 indicadores criados com sucesso
- Servidor Flask inicia sem erros em `http://localhost:5000`
- Landing page renderiza corretamente com design premium
- Fluxo de registo e login funcional com persistência JWT
- CRUD de organizações operacional
- API RESTful responde corretamente a todos os endpoints

11. Instruções de Instalação e Execução

11.1 Pré-requisitos

- Python 3.10+ instalado
- pip (gestor de pacotes Python)

11.2 Instalação

```
$ cd Fase_2/backend
$ python3 -m venv venv
$ source venv/bin/activate # Linux/Mac
$ pip install -r requirements.txt
$ cp .env.example .env # Editar com as suas configurações
$ python seed.py # Popular a base de dados
$ python run.py # Iniciar o servidor
```

Após iniciar, aceder a <http://localhost:5000> no browser.

12. Instruções para Deployment Real

Para colocar a plataforma em produção, são necessárias as seguintes alterações:

12.1 Base de Dados — Migrar para PostgreSQL:

- Instalar PostgreSQL e criar base de dados
- Alterar DATABASE_URL no .env para: postgresql://user:pass@host:5432/ailo_db
- pip install psycopg2-binary

12.2 Servidor WSGI — Gunicorn:

- Substituir `python run.py` por: `gunicorn -w 4 -b 0.0.0.0:5000 'app:create_app("production")'`

12.3 Proxy Reverso — Nginx:

- Configurar Nginx para servir ficheiros estáticos (frontend) e proxy para Gunicorn
- Configurar certificado SSL (Let's Encrypt)

12.4 Variáveis de Ambiente:

- SECRET_KEY: Gerar chave aleatória forte (ex: `python -c "import secrets; print(secrets.token_hex(32))"`)
- GEMINI_API_KEY: Obter chave em <https://aistudio.google.com/apikey>

12.5 Docker (opcional):

- Criar Dockerfile e docker-compose.yml para deployment containerizado

13. Trabalho Futuro

Funcionalidades planeadas para fases seguintes:

- Testes unitários e de integração automatizados (pytest)
- Exportação de relatórios para PDF (WeasyPrint)
- Dashboard administrativo para gestão de utilizadores
- Comparação entre avaliações da mesma organização (evolução temporal)
- Integração com LMS (Moodle) via xAPI/LTI
- Catálogo expandido de ferramentas IA com filtragem inteligente
- Benchmarking setorial (comparação entre organizações do mesmo setor)
- Aplicação mobile (PWA)

—— Fim do documento ——

Gerado automaticamente pela plataforma AILO · Maio 2026